

INFM-PROJ-ML

EFFICIENT METHODS FOR MACHINE LEARNING

at *uni-hamburg*

with *Prof. Dr. Sören Laue* in the SoSe 2024

Report by

Flavia Macovei Arian Urdu

7610738 7158460

FACULTY
OF MATHEMATICS, INFORMATICS
AND NATURAL SCIENCES

Abstract

This report explores strategies for enhancing the efficiency of natural language processing under constraints of limited training data and computational resources. Traditionally, “bigger” language models, i.e. models with more parameters produce better predictions than their smaller counterparts. This approach of scaling up models usually involves large datasets and powerful hardware, both of which can be expensive. The aim of this report is to analyse methods of optimisation that create a powerful model while requiring less data and processing power. This quality-over-quantity approach calls for high-caliber data in a distinct area. The dataset chosen for this work is the TinyStories corpus by Eldan and Li which is advantageous due to its fusion of diversification and linguistic simplicity. Among the optimisation techniques used in the model training process are thorough data preprocessing, a custom-made tokeniser and a curriculum learning agenda. In this report, these techniques are described and analysed with respect to their influence on training efficiency. In addition, the evaluation pipeline used in this process is demonstrated and justified. Our findings indicate that when designing a language model, size is not the only factor that can raise prediction quality and by implementing various optimisation techniques, small language models can generate remarkably coherent text.

1 Introduction

A system which seeks to reproduce natural language has to consider a multitude of attributes: aside from providing an appropriate vocabulary, words need to be strung together coherently and follow the appropriate grammar. Each sentence observes its intrinsic logic and successive sentences need to have an underlying correlation that is coherent. In the context of natural language processing, it appears intuitive that a language model of higher complexity can fulfill these criteria better.

In this report, we consider alternative methods for improving the efficiency of a model and aim to maximise prediction quality with strict constraints on dataset size and hardware power. More specifically, the goal of this project is to develop a small language model that is trained on a curated, small-scale dataset and whose training process can be conducted on a standard machine. The limited size of the dataset entails a need for specialisation of the prediction domain, so we aim to achieve high prediction accuracy in a specific target area. The model is to be designed in a way that it takes an input prompt in text form and predicts its continuation.

Eldan and Li pose a similar question to this report’s objective in their paper which first introduced the TinyStories dataset: “Language models ... often struggle to produce coherent and fluent text when they are small. ... This raises the question of whether the emergence of the ability to produce coherent English text only occurs at larger scales.” [?] The TinyStories dataset consists of roughly 2 million short stories generated with GPT-3.5 and GPT-4 by OpenAI in simple English and lends itself to be used in the scope of this project. Eldan and Li argue that coherent natural language prediction has to meet two criteria: it should fit both the content and the syntax of its context.

The first suggests a form of *logical reasoning* ability of the model. Take for an example the sentence “Jessica was cold so she put on ...”. The expected completion generated by a language model should relate to a warm item of clothing. The second implies the ability to use proper grammar: not any phrase that relates to clothing can be used as a continuation of the above prompt, but only those that adhere to *grammatical and syntactical rules* of the English language.

Building on these criteria, the objectives of this report are as follows:

Reproducing stories

The model should be able to generate stories similar to those of its training set given a short prompt or no prompt. They should resemble the training stories in length, language and plot complexity. A model which achieves this objective demonstrates the ability to string words together coherently, fulfilling the *grammatical and syntactical requirement*. Additionally, it is desired that the model display a certain degree of creativity and not directly replicate its training data.

Logical inference

Secondly, a successful model is expected to be able to generate a prediction that fits the logic and content of its context. This comes into play when the input prompt is longer and provides more context, thereby restricting the realm of feasible predictions. This objective feeds into the requirement for *logical conformity* mentioned above.

Competitive with leading models

Finally, the model outlined in this project is desired to demonstrate competitive performance when compared to leading benchmarks. Specifically, it should perform as well as or better than the models presented in the TinyStories paper or other notable models on selected evaluation metrics. This comparison is an indication of the model's overall capability and of the validity of the methods presented in this report.

2 The Dataset

The TinyStories dataset introduced by Eldan and Li [?] comprises a little over 2 million stories of about 175 words length on average. They are made up of roughly 17,000 unique words.¹² For reference, the average adult has a vocabulary between 25,000 and 35,000 words. [?] Children tend to acquire an extensive vocabulary later in their linguistic development, after they already have solid foundations in grammar and reasoning. This is the rationale behind this dataset, which aims to incorporate all the elementary components of the English language without introducing excessive complexity.

The stories are generated using the GPT-3.5 and GPT-4 models and mimic children's stories with a target age of 3-4.

Each story must meet the following requirements according to the authors [?]:

- it includes three words randomly chosen from a set of 1500, representing a child's vocabulary
- the format and content follows that of a standard short story intended for children
- from a range of features such as dialogue or moral value, the story must incorporate a subset

These components ensure a definite dataset in a distinct target area but which still retains sufficient data diversification for comprehensive training. The prose nature of the data also aligns well with the core objective of language models, which is to process natural language.

¹number obtained by counting all words from the dataset that are also found in the nltk corpus of words of the English language

²see appendix 7.1 for exact numbers

It is worth mentioning that the model designed in this report relies on this single data source for its training. While this approach might be less common, it is well-suited to this scenario because our goal is to generate in a restricted target area that is accurately represented by the data used. Moreover, using a sole source of data feeds into the need for simplicity and allows for a more streamlined process of development.

3 The Model

Building on the transformer architecture, the model developed for this project integrates various optimisation procedures such as thorough data preprocessing or curriculum learning. These optimisations are aimed at enhancing the model's efficiency and accuracy, particularly in generating coherent text within the targeted domain. Additionally, the model's performance is monitored through a set of evaluation metrics, ensuring that it meets the predefined benchmarks. The following section provides a detailed breakdown of these techniques and their impact on the model's performance.

Architecture

The training and prediction process of the model occurs using a transformer architecture. Transformers are a type of neural network architecture which process sequential data using self-attention mechanisms. This allows them to consider the importance of each part of the input and capture complex dependencies within it. For this reason, transformers are particularly effective for natural language processing tasks.

A 7-layer model proved to be the optimal balance between low model complexity and prediction accuracy. This configuration provides sufficient depth to capture the patterns present in the data compared to shallower models. Those struggle to generalise on the seen data and lead to results with an inconsistent flow of text as well as nonsensical words.³ Deeper models not only require more memory and processing power, they also produce results of only marginally better prediction quality.

Two embedding layers transform the input data into numerical format: A *token embedding table* is used to convert the input tokens into vector representations and a layer using a *position embedding table* encodes the position of each token in the sequence.

Self-attention layers are the core component of the transformer model. They allow it to understand the dependencies within the input sequence by weighing the relation of each pair of tokens. The model utilises 8 attention heads which perform the following functions: each head projects the input sequence into query, key, and value representations, each with outputs of dimension 8. Additionally, a dropout with probability 0.3 is performed. This final step aims to prevent overfitting by randomly deactivating a fraction of the attention weights, allowing the model to become more robust against overfitting.

Following on from there, *feed-forward* layers apply linear transformations and non-linear activation to the output from the preceding self-attention layer. The first linear transformation increases the dimensionality of the input's representation to 256 features. Subsequently, a ReLU (Rectified Linear Unit) function is applied, introducing non-linearity into the model. The features are then reduced back to their original number of 64, followed by a dropout with probability of 0.3 in these layers also.

³see appendix 7.2 for a text generated with a model of only 6 layers

Parameters

The number of parameters in a model primarily influences its ability to represent patterns in the data. The more parameters a model has, the better it can capture elaborate relationships and details. The term “model size” typically refers to the number of parameters within the model.

One factor which directly influences the number of parameters is the embedding size. This is a crucial parameter which determines the dimensionality of the space in which tokens are represented as vectors. Higher-dimensional vectors allow for more nuanced representations, but also increase the computational power needed for training and prediction. From training models with different embedding sizes, we can see that the influence of embedding size on running time is approximately linear. While we explored models with varying embedding sizes, a value of 256 was determined to be optimal for the purposes of this report. It constitutes the fitting balance between prediction quality and processing requirements.

With this embedding size, the final model comprises approximately 7.7 million parameters. The authors of the TinyStories paper developed models of different sizes between 1 million and 33 million parameters. Compared to those, the resulting size of our model is on the lower end of the spectrum which aligns with the goal for simplicity.

The optimiser used for this model is AdamW (Adaptive Moment Estimation - Weight Decay). The original Adam optimiser supports adaptive learning rates for each parameter by maintaining a running average of the gradients and squared gradients. [?] Additionally, the AdamW optimisation algorithm applies weight decay directly to the parameters after the gradient update, which is useful in particular for transformer models.

This optimiser was used in combination with a linear learning rate schedule. From an initial value of $3 \cdot 10^{-4}$, the learning rate is linearly reduced to 98.33% of the previous value in each iteration.

We used *cross entropy loss* as the target function that is to be minimised during training:

$$l(x, y) = \{l_1, \dots, l_N\}^\top,$$

$$l_n = -w_{y_n} \cdot \log \left(\frac{\exp(x_{n,y_n})}{\sum_{c=1}^C \exp(x_{n,c})} \right) \text{ for } 1 \leq n \leq N$$

where x is the input,
 y is the target,
 w is the weight,
 C is the number of classes and
 N spans the minibatch dimension
as well as $d_1, \dots, d_k$ for the k-dimensional case

Data

The dataset used in the training process consists of short stories from the TinyStories collection described above. While the raw data is highly valuable, it contains minor inconsistencies due to its generation process with a large language model, necessitating some preprocessing to ensure optimal quality for this use case.

The elementary data transformation process consisted of cleaning up those inconsistencies. Empty stories were deleted and through usage of a *set* data structure, exact duplicates were removed. Subsequently, we replaced

falsely encoded symbols such as “Ã©” with their corresponding UTF-8 representation, in this case “é”.⁴ In some instances this was not possible, and the story retained special characters despite replacing the most common encodings. Those stories were removed entirely.

This cleaning process resulted in a reduction to about 85% of the original data. The working dataset contains roughly 1.8 million data points, the number of unique words remains largely unchanged.⁵ It might appear as though this reduction performed on an already comparatively small dataset is so strong as to hinder a meaningful learning process. However, it should be pointed out that this procedure was carried out with minimal alterations, aiming to preserve the integrity of the original data. Any data that was deleted was not viable for use, therefore the reduction of *functional* data is significantly smaller. Taking this into account, the smaller size is reasonable considering this cleaning process provides high-quality data.

A more sophisticated step in the preprocessing pipeline was the creation of a three-way split on the training data for usage in the curriculum learning process which is described in detail in a later section. Such a split necessitated a ranking system for the stories resulting in a total ordering. The curriculum would then be created by sorting the stories according to their scores and dividing the resulting selection into three segments. The close connection between training data and prediction purpose suggests that the same scoring technique be used for ranking the training data as for evaluating the model output. This also has the advantage that only one scoring pipeline is implemented. The exact composition of this pipeline is examined in the *performance* section. In short, each story is passed to a large language model along with the instruction to give it a rating between 1 and 3 for its grammar, spelling, and consistency. The idea behind this automated evaluation pipeline comes from Eldan and Li, who used it to streamline the scoring process of the outputs generated by their models. [?]

Creating this total ordering on the training data was a complex process that ran for multiple days on a powerful server. Despite the project’s emphasis on minimising hardware demands, this one-time computational investment was a necessary trade-off to facilitate optimisation at later stages.

Another important consideration when working with natural language is the choice of tokeniser. The *BasicEnglish* tokeniser by *torchtext* provides a good starting point for initial experiments, but proves too complex for our scope. Any large tokeniser requires more calculations and therefore induces longer processing times during training.

Instead, we wrote a custom tokeniser to optimally handle the needs of this model. It is based on the byte-pair encoding (bpe) format and contains 4096 tokens in its vocabulary. Bpe tokenisers are beneficial in this use case compared to other types such as WordPiece or Unigram tokenisers because of their flexibility regarding out-of-vocabulary words. WordPiece and Unigram tokenisers attempt to split unknown words into sub-words contained in their vocabulary and may use a special *<unknown>* token if they are unsuccessful. Bpe tokenisers more effectively handle out-of-vocabulary words by breaking it down into subword components that exist in its vocabulary, ensuring that no part of the word is left unrepresented. This feature was critical for the choice of tokeniser, considering the chosen vocabulary size is small compared to the number of words appearing in the training data.

To further customise the tokeniser to this project’s needs, it was trained on the TinyStories dataset, thereby providing that all tokens present in its vocabulary contribute to the learning process and no space is lost.

⁴see appendix 7.3 for a full list of encoding replacements.

⁵see appendix 7.1 for exact numbers

Training

With all these features in place, the training process involved iteratively refining the model's parameters based on the data it is exposed to. The model was trained using a batch size of 32 over the course of 20 epochs.

Keeping the batch size at a moderately low level provided that the process remain accessible for execution on less powerful hardware. In order to draw on the benefits of a bigger batch size, gradient accumulation was used. This process causes gradients to be summed up over multiple, in this case two, batches before performing the update on the model's parameters. This artificially enlarges the batch size and allows for more stable gradient updates.

One of the key features of this project is the implementation of curriculum learning in conjunction with layer freezing. In preparation, a three-way split of the training data was constructed. The first split consists of the stories which scored the highest in the rating and will be called *top-level* data. The second split contains the stories that reached mid-range scores and the third split the remaining stories that scored the lowest. They will be referred to as *mid-level* and *low-level* data, respectively. The curriculum learning process entails exposing the model only to the *top-level* split first, then gradually adding the *mid-level* and *low-level* data as training progresses so that for the final iterations, the model uses the complete dataset. Combining this with a layer freezing schedule results in the following process:

1. for the first 5 iterations, training is conducted on *top-level* data only and utilises all available parameters.
2. after this phase, a portion of the layers closest to the input, in our case the first 4, are frozen and can no longer be updated during training. In the second phase, the model is trained on *top-level* and *mid-level* data, while only using the highest 3 layers.
3. During the final phase, the entire dataset is made available to the model and all layers that were previously frozen are gradually unfrozen and can be updated again.

The advantage of using this strategy is that in the early stages of training, the model learns from the best data which the collection has to offer. This knowledge is then blocked off and later iterations cannot pollute it. Finally, all layers are progressively unfrozen, allowing the model to gain a comprehensive picture of the data.

Performance

Given the specific objectives of this report, selecting the appropriate performance measures is critical for accurately evaluating the model's performance. In a section above, the loss function that is minimised during training was already discussed. This section deals with the scoring of a model after its training process has been completed. After training each model, we evaluated its performance by generating text and gathering ROUGE-L score, BLEU scores against both training data and validation data, along with two custom scores for the objectives 1 and 2.

The ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation - Longest common subsequence) score measures the longest common subsequence between the generated text and a reference text. Higher ROUGE-L scores indicate that a model has a good ability of capturing the general flow of the text, since the logical structure of

ideas is retained. Figure1 shows the correlation between the final value of the loss function at the end of a model's training and their attained ROUGE-L score.

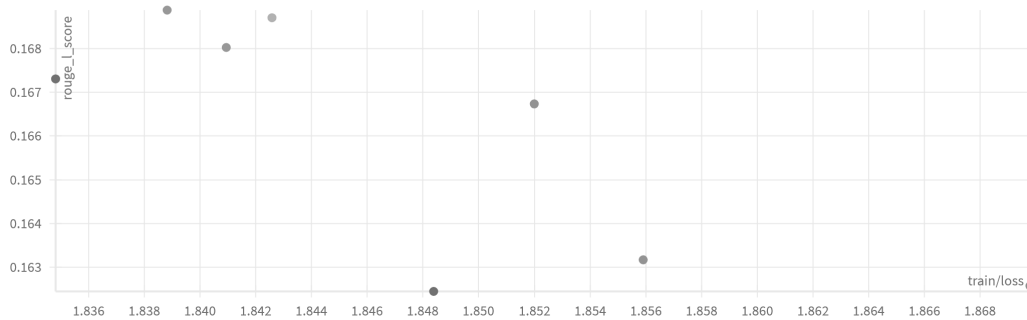


Figure 1: ROURE-L score plotted against final value of loss function on training data

It is evident that lower loss values result in a higher ROUGE-L score. The absolute values of around 0.165 might appear low compared to common benchmarks, however it should also be considered that text generation occurs in a very specific area which might be an explanation for the divergence of the scores. [?] In any event, this figure is confirmation that the chosen loss function serves the purpose of this report.

A similar case can be made using the BLEU scores on training and validation data. The BLEU (Bilingual Evaluation Understudy) score was initially designed for translations and works by measuring the overlap between the machine generated text and one or more references. In the scope of this report, it serves as an indication of how well the text generated by the model matches the training or validation data, but more importantly very high BLEU scores are a signal of overfitting. [?]

In Figure 2, the BLEU scores compared to training and validation data are plotted against the final values on the training loss and validation loss, respectively.

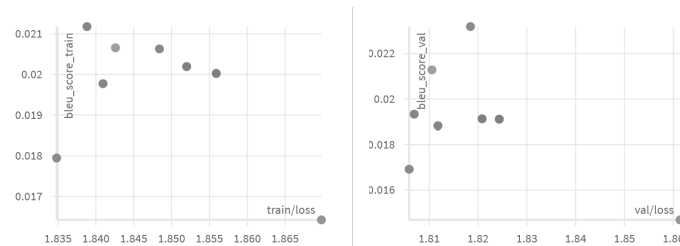


Figure 2: left: BLEU score evaluated on training data plotted against final value of loss function on training data; right: BLEU score evaluated on validation data plotted against final value of loss function on validation data

Again, lower loss values correlate with higher BLEU scores, backing the choice of loss function. But as before, the absolute values are somewhat low. [?] On the one hand, this can be a consequence of the creative nature of the task, as with the ROUGE-L score. On the other hand, it is important to note that the models depicted in these graphics generated coherent and comprehensible stories. To combat this discrepancy, we constructed two custom-made evaluation metrics that are more in line with the goals of this project.

The *stories eval* metric uses a request to the llama3 language model to measure how well the first objective is

achieved. From a randomly selected story in the training data, up to 3 words are taken and passed to the model to be scored. The generated text is then given to the llama3 language model along with the following instructions:

Please evaluate the following story based on the following three categories: Grammar, Spelling, and Consistency. Each category should be scored with an integer between 1 and 3, 1 being the worst and 3 the best. When rating a category, disregard all other errors except for those relating to that category. The output should be in the format of three numbers separated by colons, for example, '2:1:3'. Don't output any numbers below 1 or greater than 3! Output the scores only in the following format: <Grammar score>:<Spelling score>:<Consistency score>. Output nothing but the scores, no additional text or explanation.

Note that this system prompt is adamant on the format of the output. This ensures that the result received back from the llama3 model is reliably conform and can be further processed in an automated manner. The final score is obtained by performing this step multiple times and balancing the averages of the three ratings. In contrast to the ROUGE-L and BLEU metrics, the *Stories eval* metric is more aligned with the first objective of this report, which is generating coherent stories. By directly asking a language model to simulate the role of a teacher scoring a student's text, higher rating values become more representative of the quality of the generated text. Figure 3 shows the relation between the final value of the loss function and the value the corresponding model scored on *stories eval*.

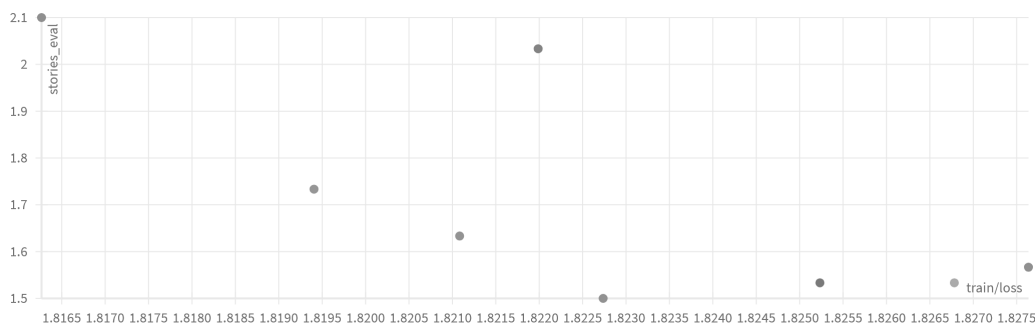


Figure 3: *stories eval* score plotted against final value of loss function on training data

On a side note, *stories eval* is the same metric that was used for ranking the stories in the training dataset.

Similarly to a custom metric for the first objective, another one was devised for the second, which is logical inference. For this, the *logic eval*, the prompts are longer. Here, the prompts passed to the model to be scored are longer and contain a logical premise along with an example answer. One such combination prompt is:

Prompt: *Michael was very tired after the game, so he went to*

Example response: *sleep*

From a compilation of 100 such prompts, one is selected randomly and presented to the model. It's response along with the following system instructions are then passed to the llama3 language model:

You will be given a prompt, an expected response, and a response generated by a language model. Your task is to evaluate how logically consistent and appropriate the generated response is compared to the expected response.

Please provide a score between 1 and 3: 1 is bad, 2 okay, and 3 good.

Don't output any numbers below 1 or greater than 3! Output nothing but the score, no additional text or explanation.

As before, it is assured that the result has the correct format, so that this evaluation can be performed multiple times and the final score calculated as the average of these runs. *Logic eval* measures how well the model can perform logical inference from a longer prompt. In Figure 4, it is shown how this metric relates to the loss functions.

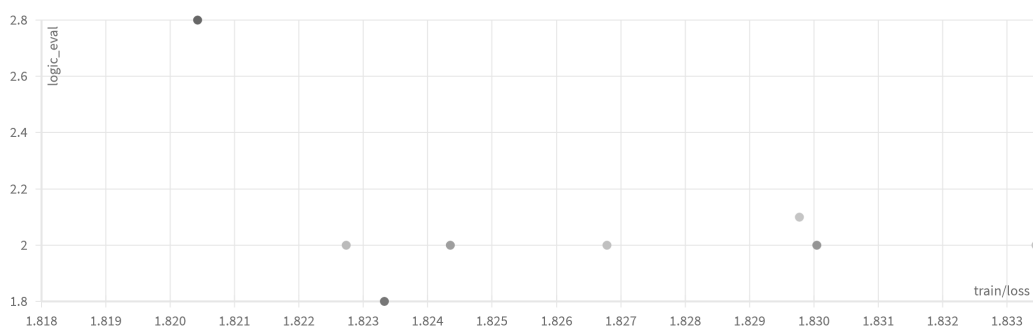


Figure 4: *logic eval* score plotted against final value of loss function on training data

With these two additional metrics, the models resulting from the training process can more accurately be evaluated with respect to the mail goals of this project. By extension, they also served in the process of selecting and refining any optimisation techniques employed.

4 Results

The final framework represents the combination of all the measures described above and can be used to train models of various sizes. Below is a table showcasing the abilities of four such models to complete a short sentence. All the models presented here are trained using our structure and arranged by their size.

→ table

Additionally, we selected the model with 7.7 million parameters as a representative for this report. The reason for this selection is that the model with 16 million parameters offers only marginal improvements in prediction quality, but at the cost of doubling the parameter count. Conversely, the 4 million parameter model demonstrates a significant reduction in performance. The chosen model scores 2.1 on the *stories eval* metric and 2.9 on the *logic eval* metric.

In line with those results, text generated by this model is coherent and follows a logical structure. When given a short input prompt it continues it to form a short story. Names and pronouns are kept consistent throughout, and the plot is intelligible. For example, if an object is used in the story, it is first introduced. Moreover, the model

can use the context of preceding text and complete sentences in a logically sensible manner.

If the input passed to the model is an out-of-vocabulary word but one that follows the structure of a word, the model can plausibly incorporate it into its story. Take for an example the input prompt “Obama”. This is neither contained in the tokeniser’s vocabulary, neither does it appear in the training data. A continuation of this prompt, generated by our model is:

was so excited to go out and play. She grabbed her bag and put her clothes inside,
and then she went to the park. When she got there, she saw a big, tall slide.
She wanted to try it out, so she ran up to it and started to climb.

A more challenging test of the model’s abilities is an out-of-vocabulary word that does not follow typical word structure, such as a random keysmash or a sequence of numbers. If the model receives a random string of letters such as “ASDFJDSIHSD”, here is an example of how it completes it:

!”, he said “I’m getting a haircut!” His Mom said “That’s wonderful, D”

which appears to be a sensible continuation. Entering a sequence of numbers does not always yield favourable results, and the model only sometimes continues on with a realistic phrase such as “years old”, otherwise it interprets the sequence as a name or outputs a different unsuitable continuation.

The nature of this project’s objectives would suggest that the generation of stories necessitates a certain degree of creativity while the deduction from a logical premise requires a more restrictive approach. Temperature, which can be seen as an indicator of creativity, is a parameter used in text generation that influences how confident the model needs to be in the next predicted token. From the list of possible subsequent tokens and their corresponding likelihood values, a high temperature allows the selection of a less likely one, while a low temperature restricts the selection to only the most likely next tokens.

Intuitively, it would be expected that a higher temperature yields better results when generating stories and a lower temperature is better fitting for a logical inference task. Experiments with the model show that, unexpectedly, the opposite holds true.

Setting the temperature to a value higher than 1 when generating stories leads to inconsistencies or “too much creativity”. For instance, the names of the characters change throughout the story or the location changes without being introduced.⁵ Values between 0.4 and 0.6 proved optimal for meaningful text generation. On the other hand, when the temperature is too low and the model is given the task of logical deduction, it struggles to find the correct answer.⁵ With the temperature set to a value between 0.6 and 0.7, we recorded the best results for the second objective.

Among the methods that were employed for optimisation, preprocessing was one of the most effective. It is important to note that when using comparatively little data, the quality of that data is all the more significant. Cleaning up any inconsistencies allowed the model to only learn from favourable stories and thereby produce better predictions. Moreover, creating the three-way split of the training set constituted an important preparation for the curriculum learning process.

⁵see appendix 7.4 and appendix 7.5 for examples for both objectives with optimal and sub-optimal temperature values

Another impactful development was the implementation of a custom tokeniser. This allowed us to specifically define the size and content of the model's vocabulary. By extent, the resulting model is smaller in size while retaining the same prediction quality.

Finally, the success of this project hinges on the implementation of a curriculum learning schedule. For a scope as restricted as this report's, building a foundational understanding before adding data complexity is a powerful tool in alleviating the training process. The schedule adds a structure to the data available to the model and compels it to focus on the highest quality data before expanding its knowledge to the full dataset. By using curriculum learning together with layer freezing, it was assured that the knowledge acquired in the early stages is not corrupted by later epochs.

The graphical processing unit employed for the training process of our final model is an NVIDIA RTX 2080 Ti and took between 5 and 6 hours to complete. This running time underscores the efficiency and accessibility of the model and aligns with our goal to ensure that the training process can be feasibly executed on standard hardware.

5 Future Work

While the development so far has already yielded notable results, demonstrating that the employment of optimisation techniques facilitates language models to predict coherent text with smaller sizes, there remain several opportunities for further exploration.

To begin with, the loss function utilised can be extended to incorporate a cosine similarity term. This adjustment would further aid the model in embedding tokens that are similar in meaning into close representations, potentially increasing its capability to comprehend the essence of its training data. First explorations of this feature proved impractical, as they drastically increased processing time. However, the expected benefits of an implementation of a cosine similarity term suggest that if its computational complexity can be minimised, the resulting models would show better performance at smaller sizes.

Secondly, we presume that the potential of the layer freezing method was not fully exhausted within the scope of this report. Further experimentation might reveal that there exists an optimal combination of freezing depth and unfreezing schedule. Finding this combination takes considerable effort and is therefore left for a future time.

Lastly, the current configuration of model does not carry the ability to end its generation sequence. - longest story + padding This suggests that the model would learn this pattern and eventually indicate the end of its generated sequence by outputting repeating padding tokens. This however is not the case and it is desired that the source of this inconsistency is identified and erased, such that the model might learn when to stop generating new text.

6 Conclusion

At the start of the report, we introduced our three objectives that this project aimed to reach: reproducing stories, logical inference, and competitiveness with leading models. Through the implementation of a series of optimisation procedures, we developed a framework which can be customised to produce models of different parameter counts with corresponding degrees of prediction quality.

The development process consisted of a valuable data preprocessing step where we not only prepared the dataset for training, but also sorted and split the stories within it with regard to their usefulness. Furthermore, the implementation of a custom tokeniser enabled optimal utilisation of the encoding capacity.

The challenges associated with preprocessing the dataset while working with hardware restrictions were effectively managed, establishing the integrity and usability of the data. Additionally, secondary methods of optimising the training process such as gradient accumulation and refining the learning rate schedule contributed significant advancements.

The most influential optimisation technique employed was the use of a curriculum learning schedule. By precisely manipulating the data available to the model, we were able to have greater control over the training process. Given the limited size of the dataset, maintaining high quality becomes even more crucial and this process ensured that the most valuable data was paired with the most significant part of training.

Reproduction of stories that resemble the training data was achieved through a series of fine-tuning steps. From the TinyStories dataset, the model has learned to obey grammatical and syntactical rules of the English language and it is able to combine sentences in a sensible manner. Our first objective, which is to generate stories similar to those present in the TinyStories collection, can therefore be viewed as attained.

More notably, the model has learned complex logical dependencies and is able to perform deduction when given a reasoning task. This can be observed in particular through the high score on the *logic eval* metric of our final model. We can assert that our second objective was achieved with the highest degree of success.

Our third objective stipulated that the model developed in this project be competitive with leading models, principally those presented in the TinyStories paper. [?] When comparing the those with our models of comparable size, it becomes apparent that they show promising results and can regard the final objective as substantially advanced towards achievement.

Our results demonstrate that the optimisation methods employed in this report significantly enhance the performance of models with relatively few parameters, enabling them to learn from limited data and generate coherent text. The influence of these techniques is apparent in our models' ability to generate meaningful output. Additionally, the successful execution of this project on standard hardware resources underscores their efficiency and practicality. These findings indicate that additional exploration of optimisation strategies could yield greater improvements in model performance.

7 Appendix

7.1 Dataset Factsheet

	before preprocessing	after preprocessing
number of stories	2,141,709	1,798,739
average length in words	175.4	174.3
number of unique words	17177	17160

7.2 Exemplary Text Generated With a 6-Layer Model

She was so happy. One day she put her scirs around the store and decided to enjoy the board. She wanted to buy the coffeeer, especially thought of silly pieces.⁶⁹⁵ When her applve, it started to smell. She was so happy that she thanked it for help. But the giant man smiled Mommy smiled and gave her a nice pictures. The ciologized spoon tasted them strong and more dess every time. Mommy thanked the card and promised to permare their fun jach on the phone. So, she asked her mom to say anything is wrong. Donn saw a tiny dinosaur poke. She drew it's patteries and happy from her mouth. She also dragged its leaf, and said no!

7.3 UTF-8 Replacements

The following table shows the combinations of special characters found in the raw TinyStories dataset and their corresponding UTF-8 representation below.

â€~	â€š	â€?	â€™	â€œ	â€“	â€”	â€¡	Ã©	Â´	Â´	Â´	Â´	Â£	Ã‰	â€	â€
,		”	,	”	-	-	...	é	,	,	,	,	£	É	”	”

7.4 Story Generation

The first part of each story is the prompt, followed by an ellipsis, followed by the text generated by our model.

Temperature: 1.2

At the park, ... Aert didn't mind. It was always sunny or hot; the park also had lots of shade and swings. Some kids liked Using ice cream, Of course, he always felt lonely and hungry. He would always sit in his trees and watch the ducks in bloom. Aert made lots of funny shapes and learned all these worms. He had so much fun! Even when it became cold, Aert found that broccoli was bad.

In the forest ... Maggie and her mom were tired but happy room had lots of gray pine trees along the walls. One particular night, they were in the forest, and Maggie then noticed something else. She saw a bottle of juice waiting for her. Stranda, the bottle was getting stuck in a dry certain pine tree next to delicious honey.

Once upon a time ... there was a green frog who loved to splash in the pond. One day, the frog was diving his head deep into the water. He swam for awhile, and swam in circles and bouncing around the pond. As he flew up high, he realized that was very dull, so he decided to keep going further and further. He was startled but smiled as he searched for his way and soon enough, he spotted a hop around near his side.

In a cozy little ... house with a big sandbox. Every day, a woman boy would go into the huge backyard. It was tidy,

and both of the towns looked so awful inside. As time left, the little boy's cap was dark. When his gold, the woman would sing his prayer for her great idea. No matter what happens, the little boy knows no matter what, the secret one is inside. From then on, the little boy's parents always searched for a hidden bow.

On a magical island ... there was a rare bird named Fiona. Fiona lived near a big tree. She liked with beauty and beauty from the tree every day. She wanted to examine each hill of the sea and the shells, so she hopped and flew until drasted the nearest float around. When she got to the bottom, she loved her new discovery.

Temperature: 1.0

At the park, ... era was very quiet. Everyone who heard to get there joined in with compliment. They thought Boters made rides and the crowd start to rot. The town was very quiet, but it was so much fun to be outside. It was perfect for everyone to have. Afterwards, Vera said goodbye to all of the people for buying. All of them had an itch on their faces.

In the forest ... a large pond was perfume. It was pink and had flowers all over it. One day, a 3 year old girl called Lucy arrived. She stepped inside and saw a blink of oil! So, she ran to the edge of the pond. The oil shapes stood in the water, and delicious smelling bitterness in her hands.

Once upon a time ... there was a friendly bird named Jack. Jack loved to fly high in the sky and sing songs. One day, he saw a big, beautiful seed in the ground. He decided to dig it with his beak and hole until he made a tunnel. Jack was so excited to race his tunnel with his friends. But when he got to the other side of the tunnel, he saw a big dog. The dog had sharp teeth and looked at Jack with curious eyes.

In a cozy little ... house, there was a little boy named Jack. Jack was three years old, and he loved to go on lots of adventures. One day, while a boy arrived, he found an incredible toy hanging on the cupboard. He was so excited and started to play with it. He made himself a few spaceship and zoomed all around the room! Jack laughed, feeling too special to keep playing with that toy.

On a magical island ... there was a shy pirate named Sally. Sally always wanted to do exciting things like to explore her ship. One day, as Sally was sailing, she met a magical fairy. "Hello Bob, what are you doing?" asked Sally. The fairy smiled and said, "I'm looking for treasure in your jewelry land."

Temperature: 0.9

At the park, ... Tim saw a big, fat cricket. He was so excited and wanted to get closer to it. He reached out and said, "Hello, little cricket." The cricket looked at Tim with its big eyes and nodded its head. Tim asked the cricket, "Why do you sit on my back?". The cricket replied, "I sit in the grass and eat some flowers." Tim reached out his hand and tucked the cricket into his pocket.

In the forest ... there was a cactus. The cactus was very small and weak, but he was strong and brave. One day the cactus decided to go for a walk in the forest. He looked around and saw a bright yellow flower. He decided to pick it up - he was so happy! The cactus held the flower close to his heart and smiled. He wanted to go back home and show it to his friends.

Once upon a time ... there was a little boy named Timmy. Timmy loved to play outside and explore the forest near his house. One day, Timmy saw a big, scary snake in the distance. He was very scared and ran towards his

mommy, but she quickly ran to the house with him. When they got on the way to his mommy, Timmy got a mask on his face. He was scared, but his mommy told him not to worry because the snake was just taking care of him.

In a cozy little ... house, there lived mom, dad, and a baby. The little baby loved making things. Today, the mommy decided to make a surprise for the baby. She took out some rags, and the little baby grabbed some flour, sugar and butter. Dad said, "Let's make a special piece of dough," Mom lit up the dough. The mommy and the baby started to make the dough.

On a magical island ... there was a little bird called Jack and he loved to sing. Every day, Jack would sing a beautiful song for birds that. One day, Jack heard a loud and good noise. He looked around and saw a black bird with sharp feathers. The bald bird had big, sharp beaks. Jack was very sad. He was about to fly away when he heard a voice. It was the voice of a fairy. "Don't be sad," said the fairy.

Temperature: 0.8

At the park, ...iolet was on the swings. She liked to swing back and forth, feeling the wind in her hair and the sun on her face. She was having so much fun! Suddenly, she saw a woman walking with a big basket full of treats. She asked Violet if she could help her. Violet was so excited and said yes.

In the forest ... there was a happy little girl called Amy. She was very curious and always wanted to explore. One day, she decided to find out. She walked for a long time until she came across an old wolf. The wolf was very friendly and he saw Amy and smiled at her. The wolf introduced himself to Amy and they became friends. Amy and the wolf played together for hours. They ran around and ate yummy honey together.

Once upon a time ... there was a little girl named Lily. She loved to play outside in the sun and watch the birds fly by. One day, Lily saw a man selling candy and wanted some. She went to the man and asked if he wanted some candy. The man said yes and gave her a lollipop. Lily was so happy and started to dance around. She twirled around and around, feeling the wind on her face. Then, she saw an old man sitting on a bench.

In a cozy little ... house, there was a little girl. Her name was Lucy and she was 3 years old. One day, Lucy found a big box in the corner of the room. She opened it and was surprised to find an old candle inside. It was so light that all of the lights around it lit up with a ring. Lucy was so excited that she began to twirl around the candle, twirling and spinning it around in her hands. She twirled and twirled and twirled.

On a magical island ... there was a little bird and a friendly parrot. They lived together in a big blue sky. One day, the parrot said: "Let's go explore the island!" The parrot happily agreed. They climbed the mountains, into the sea, and peered over mountains. They saw many different things, like trees and birds. But then, the parrot remembered something important. "Please let us go over to the island," he said.

Temperature: 0.7

At the park, ... elvet was playing in the park. She saw a big, round peach tree. It was so tasty! She wanted to take a bite, so she reached out and grabbed a small bite. It was delicious! Velvet was so excited that she started to laugh. She ran around the tree and laughed so much. Then, it was time to go home. Velvet was sad that she couldn't eat the peach.

In the forest ... there was a brave knight. He had a shiny sword and a shield. One day, he was walking through

the forest when he heard a loud noise. It was a small child, who was crying. The knight asked the child, "What's wrong?" The child said, "I hurt my knee." The knight said, "Don't worry, I'll help you."

Once upon a time ... there was a little girl named Lily. She loved to play in the park with her friends. One day, they found a big stone and decided to bury it in the sand. Lily's friend, Timmy, said, "Let's bury it in the sand!" Lily said, "No, let's bury it and see what happens." But Timmy didn't listen and buried the stone. Suddenly, a big dog came running towards them.

In a cozy little ... house, there was a very smart girl named Jen. She was three years old and liked to explore. One day, Jen went for a walk in the park. Jen saw a big tree and wanted to climb it. She was careful not to get too close to the branches. She was careful, but she was getting tired. Jen looked around the park and saw a nice spot. She found a nice spot under the tree and snuggled up.

On a magical island ... there was a unique island. It had many trees, animals and flowers. Every day, the island would bring children fun and exciting adventures. One day, a little boy named Tim went to the island. He was very excited and ran around the island until he found a big box. He peered inside and saw a beautiful necklace. He was so happy that he decided to take the necklace home. But then, the sun started to set and it started to get dark.

Temperature: 0.6

At the park, ... elvet was a very happy boy. He loved to play, run, and play with his friends. One day, Velvet noticed something strange. He saw a red button that he had never seen before. He asked his mom what it was and she said it was a little button. Velvet was so excited that he wanted to press it. His mom said that he could press the button, but only if he was careful.

In the forest ... there was a little girl called Anna. She was three years old and loved to explore. One day, Anna was walking in the forest when she saw a big pile of wood. She asked her mom, "What is this wood?" Her mom replied, "That is wood. I use it to build things. It's a very useful thing to do." Anna was very excited. She started to pick up the wood and put it in a pile.

Once upon a time ... there was a little boy named Timmy. Timmy loved to play with his toys and run around outside. One day, Timmy's mom gave him a new toy car. Timmy was so happy and he played with it all day long. But then, Timmy's mom told him that he needed to clean up his toys and put them away. Timmy didn't want to clean up, but he remembered what his mom said about cleaning up.

In a cozy little ... house, there was a little boy named Timmy. Timmy loved to play with his toys and run around outside. One day, Timmy's mom asked him to help her in the kitchen. Timmy was a little scared because he didn't know how to do it. "Mommy, I'm scared," Timmy said. "It's okay, Timmy. I'll help you. Let's put some salt in the soup."

On a magical island ... there was a little girl. She was very excited because she had never been on a boat before. She had been hoping to get there for a long time, but it was too far away. She looked around the island and saw a big tree with a big bird in it. The little girl was so excited! She ran to the tree and tried to climb it. But the tree was too high and the little girl couldn't reach it. She started to get frustrated.

Temperature: 0.5

At the park, ...iolet was playing with her friends. She was having lots of fun, but then she saw a big, scary bug. She was scared and started to cry. Her friends came over and asked her what was wrong. Violet told them about the big bug. They wanted to help her get rid of it. Violet's friends had an idea. They all got some sticks and stones and started to make a big pile of leaves.

In the forest ... there was a little boy named Jack who was very brave. He loved to play outside in the forest and explore the forest. One day, he found a big tree with lots of leaves. He wanted to climb it, but he was scared. Jack thought about how he could climb the tree, but then he remembered something his mum had told him. She said: "If you are brave and try your best, you can do it."

Once upon a time ... there was a little girl named Lily. She loved to play with her toys and run around outside. One day, her mom told her that they were going to visit her grandma. Lily was very excited and couldn't wait to see her grandma. When they arrived, Lily saw a big building with a sign that said "Welcome" on it. She asked her mom what it meant. Her mom explained that it meant they were going to visit grandma.

In a cozy little ... house, there was a little girl named Lily. She loved to play with her toys and make up stories with them. One day, her mom asked her to help with the laundry. Lily was happy to help and started to fold the clothes. As she was folding, her mom asked her to put the clothes in the laundry basket. Lily was excited to help and started to fold the clothes. She folded the clothes into piles and put them in the basket.

On a magical island ... there was a little girl named Daisy. Daisy was only three years old and she loved to explore the island. One day, Daisy was walking along the beach when she saw something very special. It was a big box with a lock on it. She was so excited, she couldn't wait to open it. Daisy carefully opened the box and inside she found a big, shiny golden key. She was so happy and she couldn't believe her luck.

Temperature: 0.4

At the park, ... elvet was a very happy boy. He liked to play with his friends and have fun. One day, Velvet saw a big, red ball. He wanted to play with it. He ran to the ball and tried to kick it. But the ball was too high. Velvet was sad. He wanted to get the ball back. Then, a little girl came by.

In the forest ... there was a little girl named Jane. She was only three years old and loved to explore. One day, she went on an adventure. Jane walked through the forest and soon she came across a big bush. She was curious and wanted to know what was inside. She slowly opened the bush and found a beautiful crystal. Jane was so excited and she wanted to show her mom.

Once upon a time ... there was a little girl named Lily. She loved to play with her toys and draw pictures. One day, Lily's mom asked her to clean up her toys. Lily didn't want to clean up, but her mom said it was important to keep her room clean. Lily started to clean up her toys, but her mom said it was important to keep her room clean. Lily didn't want to clean up, but she knew it was important to keep her room clean.

In a cozy little ... house, there was a little girl named Lily. She loved to play with her toys and her favorite toy was a teddy bear named Teddy. One day, Lily's mom asked her to clean up her toys and put them away. Lily didn't want to clean up, but she knew she had to listen to her mom. Lily started to pick up her toys and put them away.

On a magical island ... there was a little girl named Lulu. She was only three years old, but she was very brave. One day, Lulu decided to explore the island. She asked her mom if she could go. "Yes, you can go," said her mom. So Lulu and her mom went to the island. Lulu was so excited! She saw a big, beautiful lake with ducks swimming in it.

Temperature: 0.3

At the park, ... elvet was a very happy little girl. She loved to play with her friends and have fun. One day, Velvet saw a big, shiny ball in the grass. She ran over to it and picked it up. She was so excited! Velvet wanted to play with the ball, but it was too high up. She tried to reach it but it was too high. She asked her friends for help.

In the forest ... there was a little girl named Sarah. She was very curious and wanted to explore the forest. One day, she decided to go for a walk. As she walked, she noticed a big, green tree. She wanted to climb it, but it was too high. Just then, a voice said, "Be careful, Sarah. Don't climb too high." Sarah looked around and saw a wise old owl sitting on a branch.

Once upon a time ... there was a little girl named Lily. She loved to play outside in the sun. One day, she saw a big, scary dog in the park. The dog was barking loudly and Lily was scared. Lily's mom saw what was happening and told her not to be afraid. She said the dog was just looking for food and not going to hurt her. Lily felt better and said, "I'm sorry, mommy. I didn't mean to scare you."

In a cozy little ... house, there was a little girl named Lily. She loved to play with her toys and her favorite toy was a teddy bear named Teddy. One day, Lily's mom asked her to help with the laundry. Lily was very excited to help, so she picked up a big basket and started to put the clothes in the basket. As she was putting the clothes in the basket, she noticed that her mom was getting a little tired. "Mommy, I'm tired," Lily said.

On a magical island ... there was a little girl named Daisy. Daisy was very excited because she was going to visit her grandma. She was going to visit her grandma and she had never been to a island before. When Daisy arrived, she was amazed by all the wonderful things she saw. There were trees, flowers, and even a big pond. She also saw a big pond with ducks swimming in it. Daisy was so excited.

Temperature: 0.1

At the park, ... elvet was a very happy little girl. She loved to play with her friends and have fun. One day, Velvet saw a big, red ball. She wanted to play with it, but it was too high for her to reach. Velvet asked her mom for help. Her mom said, "Let's get a ladder and you can get the ball."

In the forest ... there was a little girl named Lucy. She was very curious and loved to explore. One day, she found a big, shiny rock. She was so excited and wanted to take it home. Lucy asked her mom, "Can I keep this rock?" Her mom said, "Yes, you can keep it." Lucy was so happy and she kept the rock in her room. She kept it safe and she kept it safe.

Once upon a time ... there was a little girl named Lily. She loved to play outside in the sunshine. One day, she saw a big, scary dog in the park. The dog looked very scary and Lily was scared. Lily's mom came to her and said, "Don't worry, Lily. The dog is friendly. He won't hurt you." Lily felt better and said, "Thank you, Mommy. I like the dog."

In a cozy little ... house, there was a little girl named Lily. She loved to play outside in the sunshine. One day, she saw a big, scary dog in her backyard. The dog was barking and running towards her. Lily was scared, but she remembered her mom's words: "Don't be afraid, Lily. The dog is friendly." Lily walked up to the dog and said, "Hello, little dog. What's your name?" The dog barked back and Lily felt better.

On a magical island ... there was a little girl named Lily. She loved to play in the sand and look at the pretty flowers. One day, she saw a big, shiny rock on the ground. She picked it up and showed it to her mom. "Look, Mommy! I found a rock!" Lily said. "That's great, Lily. But be careful, it's very sharp. You might hurt yourself or someone else," her mom said.

7.5 Logical Inference

First comes the prompt in the form of a logical premise, followed by an ellipsis, followed by the response generated by our model.

Temperature: 1.2

John was thirsty, so he opened a bottle of... fresh oil and made a nice salad! He.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some... salt.

Mike was preparing for his exam and needed to focus, so he turned off his... modern autumn.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the... apple.

David went to bed early yesterday because he had to wake up... early before dark.

Chris was playing football when he hurt his leg and had to stop... thinking.

Anna loves reading. She started a new book and couldn't stop... talking about her family.

Tom was having a party and invited all his... friends.

Jessica was cold, so she put on her... cozy coat and hat.

Michael was very tired after the game, so he went to... take a nap with his mom around his bedroom.

Temperature: 1.0

John was thirsty, so he opened a bottle of... perfume.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some... salt.

Mike was preparing for his exam and needed to focus, so he turned off his... rubber scarf, grabbed some chalk.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the... apple.

David went to bed early yesterday because he had to wake up... before tomorrow.

Chris was playing football when he hurt his leg and had to stop... He tried to stand up but he couldn't.

Anna loves reading. She started a new book and couldn't stop... moving it.

Tom was having a party and invited all his... friends to come and visit.

Jessica was cold, so she put on her... coat.

Michael was very tired after the game, so he went to... bed with his mom.

Temperature: 0.9

John was thirsty, so he opened a bottle of... milk.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some... "Will they supply us with new.

Mike was preparing for his exam and needed to focus, so he turned off his... paper and paper to start his exam.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the... apple.

David went to bed early yesterday because he had to wake up... early and eat his breakfast.

Chris was playing football when he hurt his leg and had to stop... it.

Anna loves reading. She started a new book and couldn't stop... smiling.

Tom was having a party and invited all his... friends to come.

Jessica was cold, so she put on her... warm coat and grabbed her favorite scarf.

Michael was very tired after the game, so he went to... take a nap.

Temperature: 0.8

John was thirsty, so he opened a bottle of ... perfume.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... help.

Mike was preparing for his exam and needed to focus, so he turned off his ... shoes and went outside.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... He was sad because he wanted to keep playing.

Anna loves reading. She started a new book and couldn't stop ... She reads about animals, stars, and.

Tom was having a party and invited all his ... friends to come.

Jessica was cold, so she put on her ... warm sweater.

Michael was very tired after the game, so he went to ... bed.

Temperature: 0.7

John was thirsty, so he opened a bottle of ... juice and drank it all up.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... salt.

Mike was preparing for his exam and needed to focus, so he turned off his ... shirt and mittens and set out to work.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... He didn't understand why he was hurt.

Anna loves reading. She started a new book and couldn't stop ... thinking about the words.

Tom was having a party and invited all his ... friends.

Jessica was cold, so she put on her ... warm coat.

Michael was very tired after the game, so he went to ... his bedroom to see the clock.

Temperature: 0.6

John was thirsty, so he opened a bottle of ... water.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... salt.

Mike was preparing for his exam and needed to focus, so he turned off his ... shoes and grabbed his favourite toy.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... His dad had to go to the doctor,.

Anna loves reading. She started a new book and couldn't stop ... smiling.

Tom was having a party and invited all his ... friends over.

Jessica was cold, so she put on her ... warm coat.

Michael was very tired after the game, so he went to ... sleep in his bed.

Temperature: 0.5

John was thirsty, so he opened a bottle of ... water.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... His neighbor was a nice lady who smiled at.

Mike was preparing for his exam and needed to focus, so he turned off his ... pencil and went to the farm.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... He was feeling very sad and he couldn't.

Anna loves reading. She started a new book and couldn't stop ... smiling.

Tom was having a party and invited all his ... friends to come over.

Jessica was cold, so she put on her ... warm coat.

Michael was very tired after the game, so he went to ... the park.

Temperature: 0.4

John was thirsty, so he opened a bottle of ... water.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... salt.

Mike was preparing for his exam and needed to focus, so he turned off his ... mattress and headed to school.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... He looked around and saw a big tree with.

Anna loves reading. She started a new book and couldn't stop ... smiling.

Tom was having a party and invited all his ... friends to come.

Jessica was cold, so she put on her ... warm coat and went outside to play.

Michael was very tired after the game, so he went to ... sleep.

Temperature: 0.3

John was thirsty, so he opened a bottle of ... water.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... salt.

Mike was preparing for his exam and needed to focus, so he turned off his ... bed and went to the kitchen.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... He was feeling very sad.

Anna loves reading. She started a new book and couldn't stop ... smiling.

Tom was having a party and invited all his ... friends to come.

Jessica was cold, so she put on her ... warm coat and went outside.

Michael was very tired after the game, so he went to ... his bedroom to get a nap.

Temperature: 0.1

John was thirsty, so he opened a bottle of ... water.

Emma was cooking and realized she ran out of salt, so she asked her neighbor for some ... salt.

Mike was preparing for his exam and needed to focus, so he turned off his ... bed and went to the kitchen.

Sarah had an orange and an apple in her hand. She ate the orange and then ate the ... apple.

David went to bed early yesterday because he had to wake up ... early.

Chris was playing football when he hurt his leg and had to stop ... He was very sad and he didn't know.

Anna loves reading. She started a new book and couldn't stop ... smiling.

Tom was having a party and invited all his ... friends to come and play.

Jessica was cold, so she put on her ... warm coat and went outside.

Michael was very tired after the game, so he went to ... bed.